

Matrix Function Approximation with Chebyshev Polynomials

Ava Labmeier

1 Theory

1.1 Matrix Functions and Spectral Decomposition

Let $A \in \mathbb{R}^{N \times N}$ be a symmetric matrix. By the spectral theorem, A can be factorized as

$$A = U\Lambda U^T,$$

where U is orthogonal and

$$\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_N)$$

is diagonal. The values $\lambda_1, \lambda_2, \dots, \lambda_N$ are the eigenvalues of A .

Let

$$p(x) = \sum_{k=0}^m c_k x^k$$

be a polynomial of degree m . Since

$$A^k = U\Lambda^k U^T,$$

we obtain

$$p(A) = \sum_{k=0}^m c_k A^k = \sum_{k=0}^m c_k U\Lambda^k U^T.$$

Factoring out U and U^T , this becomes

$$p(A) = U \left(\sum_{k=0}^m c_k \Lambda^k \right) U^T = U p(\Lambda) U^T.$$

Thus, applying a polynomial to a symmetric matrix is equivalent to applying the polynomial to the eigenvalues of the matrix.

1.2 Polynomial Approximation of Matrix Functions

Let $A = U\Lambda U^T$, where the eigenvalues of A lie in an interval $[a, b]$. Suppose $\{p_k\}_{k=0}^\infty$ is a sequence of polynomials that converges pointwise to a function f on $[a, b]$. Then

$$p_k(A) = Up_k(\Lambda)U^T.$$

Taking limits gives

$$f(A) = \lim_{k \rightarrow \infty} p_k(A) = U \left(\lim_{k \rightarrow \infty} p_k(\Lambda) \right) U^T = Uf(\Lambda)U^T.$$

Here,

$$f(\Lambda) = \text{diag}(f(\lambda_1), f(\lambda_2), \dots, f(\lambda_N)).$$

This definition is independent of the polynomial sequence chosen because any sequence of polynomials converging to f on $[a, b]$ must converge to the same value at each eigenvalue λ_i .

1.3 Matrix Exponentials

For a function represented by a convergent Taylor series,

$$f(x) = \sum_{k=0}^{\infty} a_k(x - c)^k,$$

the corresponding matrix function is

$$f(A) = \sum_{k=0}^{\infty} a_k(A - cI)^k.$$

In particular, for $f(x) = e^x$,

$$e^A = \sum_{k=0}^{\infty} \frac{A^k}{k!}.$$

This provides a theoretical foundation for approximating matrix functions using polynomial expansions.

2 Motivation

The goal of this project is to efficiently approximate matrix-vector products of the form

$$f(A)v,$$

where A is a large sparse matrix and v is a vector.

A direct approach would compute the eigendecomposition of A , form $f(A)$, and then multiply by v . However, eigendecomposition is computationally expensive, requiring approximately $O(N^3)$ operations for an $N \times N$ matrix. This becomes impractical when N is large.

Instead, this project uses Chebyshev polynomial approximation. If the eigenvalues of A lie in an interval $[a, b]$, the interval can be mapped to $[-1, 1]$, where Chebyshev polynomials provide an efficient approximation basis. The three-term recurrence relation allows $f(A)v$ to be approximated using only matrix-vector multiplications.

This is especially useful for sparse matrices because sparse matrix-vector products are significantly cheaper than dense eigendecomposition or explicitly forming $f(A)$.

3 Implementation

3.1 Chebyshev Polynomial Evaluation

Chebyshev polynomials of the first kind are defined by the recurrence relation

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x),$$

with

$$T_0(x) = 1, \quad T_1(x) = x.$$

This recurrence is implemented in `cheb_eval.m`. The function evaluates $T_n(x)$ for scalar or vector inputs.

3.2 Chebyshev Coefficient Computation

The Chebyshev coefficients are computed using Gauss-Chebyshev quadrature. The interval $[a, b]$ is mapped to $[-1, 1]$, and the target function is evaluated at quadrature nodes.

This produces coefficients

$$c_0, c_1, \dots, c_n$$

for the Chebyshev approximation of f . This step is implemented in `cheb_coeff.m`.

3.3 Matrix-Vector Approximation

The matrix A is transformed so that its spectrum lies in $[-1, 1]$. If the spectrum of A lies in $[a, b]$, then the transformed matrix is

$$\tilde{A} = \frac{2}{b-a}A - \frac{a+b}{b-a}I.$$

The approximation to $f(A)v$ is then computed using the Chebyshev recurrence. This avoids explicitly forming $f(A)$ and requires only repeated matrix-vector products.

This procedure is implemented in `cheb_matvec.m`.

4 Numerical Experiments

4.1 Relative Error Analysis

To evaluate accuracy, the Chebyshev approximation was compared against the exact result computed using MATLAB's `expm` function.

The experiment tested

$$N = 100, 200, \dots, 1000$$

and

$$C \in \{0.01, 0.1, 0.25, 0.5\}.$$

The polynomial degree was chosen as

$$n = CN.$$

The relative error was computed as

$$\frac{\|w_{\text{approx}} - w_{\text{true}}\|}{\|w_{\text{true}}\|}.$$

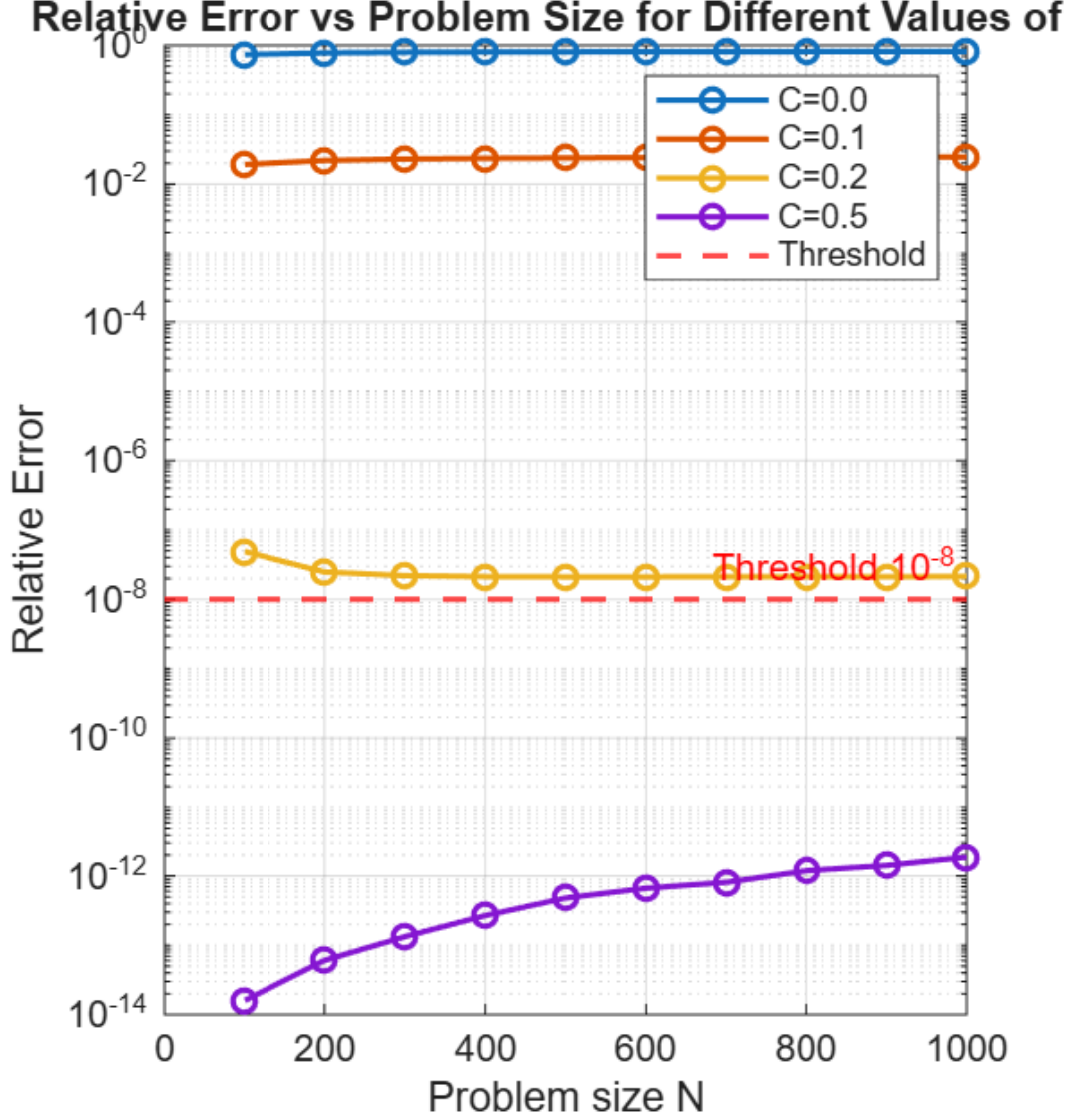


Figure 1: Relative error for different values of C as N increases.

The results show that approximation accuracy improves as the truncation parameter C increases. Smaller values such as $C = 0.01$ and $C = 0.1$ produce larger relative errors. The value $C = 0.5$ consistently achieves errors below the threshold of 10^{-8} , showing that a sufficiently large polynomial degree is necessary for high accuracy.

4.2 Runtime Analysis

Runtime experiments were conducted for

$$N = 500, 1000, \dots, 3000.$$

For each value of N , the runtime of the Chebyshev approximation method was measured. A

quadratic curve was then fit to the measured times.

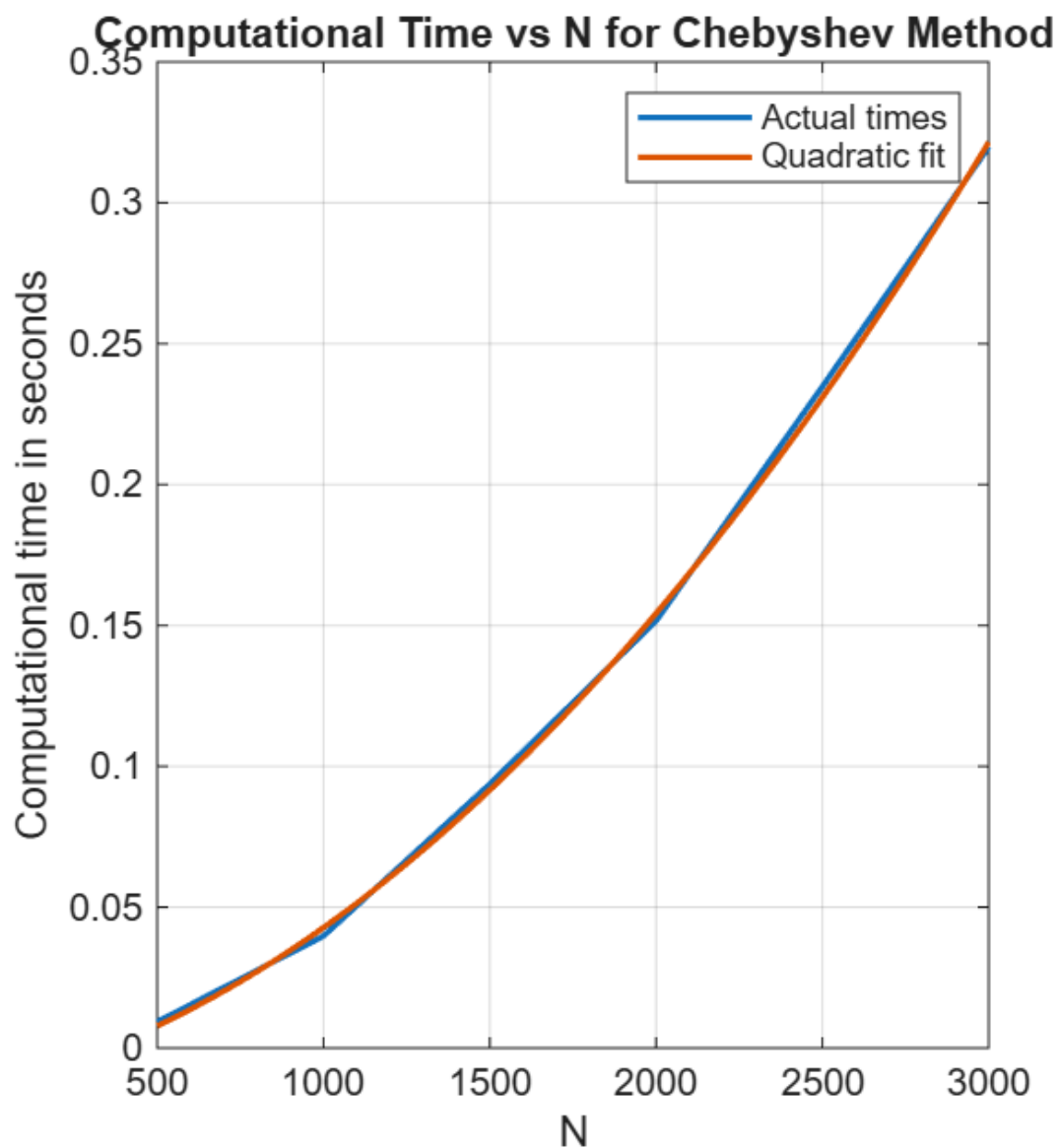


Figure 2: Computational runtime as a function of N , with a quadratic fit.

The measured runtimes closely follow the fitted quadratic curve, supporting the expected $O(N^2)$ scaling behavior of the implementation.

5 Conclusion

This project implemented a Chebyshev polynomial method for approximating matrix functions applied to vectors.

By avoiding direct eigendecomposition and relying on sparse matrix-vector products, the method provides a more efficient alternative for large sparse matrices. The numerical experiments demon-

strated that the approximation can achieve high accuracy when the polynomial degree is chosen appropriately. Runtime testing also confirmed the expected quadratic scaling behavior.

Overall, Chebyshev polynomial approximation provides a practical and scalable method for computing expressions of the form $f(A)v$ in large-scale numerical linear algebra problems.